Advanced SQL Interview Patterns (SDE 2 - Staff Level) - Comprehensive Edition

This document outlines common patterns of advanced SQL questions asked during technical interviews for mid-level to Staff engineering roles. These questions go beyond simple syntax and test for deep understanding of query logic, performance optimization, data modeling trade-offs, and data integrity in a real-world business context. Questions are categorized by difficulty to aid in progressive learning.

Pattern 1: Window Functions & Time-Series Analysis

These scenarios test your ability to perform complex calculations across related sets of rows, which is essential for analytics, reporting, and feature logic.

Easy

1. **Simple Ranking:** Find the top 5 employees with the highest salaries in the entire company.
2. **Departmental Ranking:** List employees in each department, ranked by their salary from highest to lowest.
3. **Previous and Next Value:** For each order, show the order ID, order date, and the date of the very next order placed by the same customer.
4. **Basic Running Total:** Provide a daily report of sales that includes the day's total sales and a cumulative sum of sales since the beginning of the month.
5. **Row Numbering:** Assign a unique sequential number to each user based on their sign-up date.
6. **Simple Partitioning:** What is the most expensive product in each product category?

Medium

1. **Top N in Each Group (Salary Audit):** HR is conducting a salary audit. They need a report of the top 3 earners in each department. To provide context, also include each employee's tenure (time since hire_date) and compare their salary to the department's average salary using a window function.

2. **Moving Average for Financials:** To smooth out daily revenue fluctuations, the finance team needs a 7-day moving average of sales. Given a sales table with sale_date and amount, write a query that shows the revenue for each day and its corresponding 7-day moving average.
3. **Month-over-Month Growth:** The marketing team wants to see the month-over-month percentage growth in the number of new user sign-ups.
4. **Detecting Declining Sales:** The sales team needs to be alerted about underperforming products. Write a query to find all products whose total sales have declined for two consecutive months.
5. **Identifying Gaps:** You have a table of sequential check numbers. Write a query to find the first missing check number in the sequence.
6. **Salary Quartiles:** For an internal equity report, assign each employee to a salary quartile (1 to 4) within their respective department.

Hard

1. **Consecutive Activity Streaks:** The product team wants to identify highly engaged users. Write a query to find all users who have logged in for at least 5 consecutive days. Your output should be the user_id and the start date of their first 5-day streak.
2. **Sessionization:** The analytics team needs to calculate the average duration of a user session. A session is defined as a series of events from a single user where the time between consecutive events is no more than 30 minutes. You have an events table with user_id and event_timestamp.
3. **Conditional Running Total:** Calculate a running total of transaction amounts for each customer, but have the total reset every time a transaction of type 'refund' is encountered.
4. **Islands and Gaps:** Given a table of dates when a server was online, write a query to identify the start and end dates of each continuous period of uptime (the "islands").
5. **Finding the Second Longest Streak:** From a logins table, find the user with the second-longest consecutive daily login streak.

Pattern 2: Complex Joins, Aggregations, and Self-Joins

This pattern tests your ability to synthesize information from multiple sources, correctly filter aggregated results, and find relationships within the same dataset.

Easy

1. **Manager-Employee Salary Comparison:** As part of a compensation fairness analysis, write a query to find all employees who earn more than their direct manager. The employees table contains id, name, salary, and manager_id.
2. **Finding Orphaned Records:** For data cleanup, you need to find all authors who have no books associated with them. A LEFT JOIN is a good approach here, but explain why and how you'd structure the query.
3. **Basic Aggregation with Filtering:** List all departments that have more than 10 employees.
4. **Joining Three Tables:** Get a list of customer names, the products they bought, and the date of the transaction. This requires joining customers, orders, and products.
5. **Employees in Same Location:** Write a query to find all pairs of employees who work in the same city.

Medium

1. **Product Affinity Analysis:** To improve product recommendations, marketing wants to know which products are frequently bought together. Write a query to find pairs of products that appear in the same transaction more than 100 times.
2. **Multi-hop Social Connections:** On a social network, a "friend of a friend" recommendation is powerful. Given a friendships table (user_id_1, user_id_2), write a query to find all of a specific user's "friends of friends," excluding their direct friends.
3. **Aggregation with Multiple Conditions:** Find all departments where the average salary of employees with more than 5 years of tenure is greater than $100,000.
4. **Cross-Join for Pairings:** Generate a list of all possible pairs of mentors and mentees from an employees table, excluding pairs where the person is both.
5. **Using FULL OUTER JOIN:** You have two tables, marketing_leads and sales_conversions. Find all individuals who

exist in one list but not the other, to identify leads that didn't convert and conversions that weren't tracked as leads.

Hard

1. **The "Division" Problem:** To plan a new marketing campaign, we need to identify our most versatile shoppers. Write a query to find all customers who have purchased at least one product from *every single* product category.
2. **Complex Self-Join for Event Sequencing:** You have a table of user actions with timestamps. Find all users who performed action 'B' within 10 minutes of performing action 'A'.
3. **Mutual Friendships:** In a social network friendships table, a friendship is stored as a single row (e.g., user 1 is friends with user 2). Write a query to find all pairs of users who have at least 5 mutual friends.
4. **Finding Supervisors with Overlapping Teams:** Identify pairs of managers whose teams have at least 3 employees with the same job title.
5. **Closest Match Join:** You have a table of sales and a table of promotions with start_date and end_date. For each sale, join it to the promotion that was active on the sale_date. The challenge is that a sale could fall under multiple promotions if their date ranges overlap; ensure you only link to one.

Pattern 3: CTEs (Common Table Expressions) & Recursive Queries

CTEs are crucial for writing clean, readable queries. Recursive CTEs are a key test for handling hierarchical data.

Easy

1. **Simple CTE for Aggregation:** Using a CTE, find the department with the highest average salary.
2. **Breaking Down a Join:** Rewrite a query that joins three tables (customers, orders, order_details) to calculate total revenue per customer, using a CTE to first aggregate the order_details.
3. **Filtering with a CTE:** Find all employees who work in departments that have a budget over $1 million. Use a CTE to first identify the qualifying departments.
4. **Pre-calculating a Value:** Use a CTE to calculate the overall average salary for the company, then find all employees who earn above that average.
5. **Basic Readability Improvement:** Take a complex query with multiple subqueries and refactor it to use CTEs for clarity.

Medium

1. **Multi-step Funnel Analysis:** The product team wants to analyze a user conversion funnel (e.g., view_homepage -> view_product -> add_to_cart -> purchase). Using CTEs, write a query that calculates the number of unique users who completed each step and the conversion rate from one step to the next.
2. **Calculating Lifetime Value (LTV):** The marketing team needs to calculate the LTV for each customer. First, use a CTE to aggregate total spending per customer from an orders table. Then, join this with a customers table to show the LTV alongside customer registration details.
3. **Finding Second-Degree Connections:** Using two CTEs, find the "friends of friends" for a specific user. The first CTE gets direct friends, the second gets the friends of those friends.
4. **Complex Filtering Logic:** Find the top 5 customers who have spent the most in the last 6 months on products from the 'Electronics' category. Use CTEs to isolate the relevant time frame, customers, and categories before the final aggregation.

Hard

1. **Organizational Hierarchy Traversal:** For a new company directory feature, we need to display the entire reporting structure. The employees table has employee_id, name, and manager_id. Using a recursive CTE, generate a report that shows each employee, their manager, and their level in the hierarchy (CEO is level 1). Also, show the full reporting path (e.g., "CEO->VP->Director->Employee").
2. **Finding Gaps in Sequential Data:** During a data audit, you suspect some invoice_ids are missing. The IDs are supposed to be sequential. Using a recursive CTE, generate all numbers from the minimum to the maximum invoice_id and find which ones are missing from the table.
3. **Bill of Materials (Parts Explosion):** You have a parts table where some parts are made of other sub-parts (part_id, sub_part_id, quantity). Using a recursive CTE, write a query to find all the individual base components required to build a single "car".
4. **Flight Itinerary Pathfinding:** Given a flights table (origin, destination), find a path of flights from 'New York' to 'San Francisco'. The output should be a single string representing the path, like 'JFK -> ORD -> SFO'.
5. **Cycle Detection in a Graph:** Using a recursive CTE, check if a management hierarchy contains a cycle (e.g., A manages B, and B manages A).

Pattern 4: Performance & Query Optimization

For senior and staff roles, a correct but slow query is a failed answer.

Easy

1. **SELECT *:** Why is SELECT * considered a bad practice in production code?
2. **Basic Indexing:** You have a users table and you frequently query it with WHERE email = '...'. What can you do to speed this up?
3. **WHERE vs. HAVING:** Explain the difference between the WHERE and HAVING clauses and how their placement affects a query.
4. **UNION vs. UNION ALL:** What is the difference between UNION and UNION ALL? Which is more performant and why?
5. **Slow Query on Large Table:** A simple SELECT on a large table is slow. What's the most likely cause? (Lack of index leading to a full table scan).

Medium

1. **EXPLAIN Plan Analysis:** You run EXPLAIN on a query and see it's doing a "Full Table Scan" on a large table despite an index existing on the WHERE clause column. What are some possible reasons the query optimizer is ignoring the index (e.g., low cardinality, outdated statistics, non-sargable query)?
2. **Indexing Strategy for High-Write Systems:** You are designing a feature that tracks every time a user views a product. This will generate a very high volume of writes. Reads are also frequent for a "most viewed products" page. Discuss your indexing strategy for the product_views table. What are the trade-offs?
3. **Correlated Subquery vs. JOIN:** You find a query in the codebase that uses a correlated subquery. Explain the performance implications (often row-by-row execution) and rewrite it using a JOIN or a CTE.
4. **Composite Indexes:** You have a query: SELECT * FROM orders WHERE customer_id = ? AND order_date > ?. What would be the optimal index to create for this query? Explain the importance of column order in a composite index.
5. **Covering Indexes:** What is a "covering index"? How can it dramatically improve query performance by avoiding table lookups?

Hard

1. **Diagnosing a Slow Dashboard:** A dashboard query that calculates daily active users (DAU) by joining a 500-million-row events table with a 50-million-row users table is timing out. Walk through your step-by-step process to diagnose and fix it. Discuss query rewriting, indexing, materialized views, and potential data denormalization.
2. **Parameter Sniffing:** What is parameter sniffing in SQL Server (or a similar concept in other DBs)? Describe a scenario where it could cause a query to suddenly become slow and how you would mitigate it.
3. **Pagination Performance:** The query for page 1 of a result set (OFFSET 0 LIMIT 20) is fast, but the query for page 50,000 (OFFSET 1000000 LIMIT 20) is extremely slow. Explain why this happens and describe a more performant way to implement deep pagination (e.g., keyset/cursor-based pagination).
4. **Index Merge vs. Composite Index:** A query filters on two separate columns, col_A and col_B. The database has separate indexes on each. Sometimes the optimizer performs an "Index Merge" and sometimes it's slow. Would a composite index on (col_A, col_B) be better? Why or why not?
5. **Optimizing LIKE Queries:** A query using WHERE name LIKE '%johnson%' is slow. Why can't the database use a standard B-Tree index for this? What alternative strategies could you use to enable fast text searching (e.g., Full-Text Search, different index types like GIN/GiST in Postgres)?

Pattern 5: Data Modeling & Schema Design

These questions test your architectural thinking and ability to model complex business requirements.

Easy

1. **Normalization:** What is database normalization? Briefly explain the purpose of 1NF, 2NF, and 3NF.
2. **Primary vs. Foreign Key:** Explain the difference between a primary key and a foreign key with a simple Authors and Books example.
3. **Many-to-Many Relationship:** How do you model a many-to-many relationship in a relational database? (e.g., Students and Courses).
4. **Data Types:** Why is it important to choose the correct data type for a column (e.g., INT vs VARCHAR for a phone number, TIMESTAMP vs DATETIME)?
5. **Surrogate vs. Natural Key:** What is the difference between a surrogate key and a natural key? What are the pros and cons of using a surrogate key?

Medium

1. **Inventory Management System:** Design a schema for an e-commerce inventory system. It must support multiple warehouses and track stock levels for products that have variations (e.g., a T-shirt in different sizes and colors). How do you model the relationship between a base product, its variants, and its stock level in each warehouse?
2. **"Soft Deletes":** Your application needs to support "soft deletes" (marking a record as deleted instead of actually removing it). How would you implement this? Discuss the impact on queries, indexes, and foreign key constraints.
3. **Modeling Hierarchical Data (Adjacency List):** You need to store product categories with nested sub-categories. Design the schema using the Adjacency List model (id, name, parent_id). Discuss the pros and cons of this simple approach.
4. **Denormalization:** When and why would you choose to intentionally denormalize a database schema? Provide a concrete example, such as storing a post_comment_count on a posts table. What are the drawbacks?
5. **Schema for a Blogging Platform:** Design the core tables for a simple blog: users, posts, comments, and tags. How would you link tags to posts?

Hard

1. **Ride-Sharing App:** Design the core database schema for a ride-sharing app. Sketch out the tables for rides, users (riders), drivers, and vehicles. How would you model the state of a ride (e.g., requested, accepted, in_progress, completed)? How would you handle pricing, payments, and ratings in a scalable way?
2. **User-Defined Fields (EAV vs. JSONB):** You need to design a system where users can add custom key-value fields to their profiles (like in Salesforce or JIRA). Compare and contrast two approaches: a rigid EAV (Entity-Attribute-Value) model and a more flexible approach using a JSONB/JSON column. Discuss query complexity, indexing, and performance for each.
3. **Modeling Hierarchical Data (Nested Set):** Compare the Adjacency List model to the Nested Set model for storing hierarchies. Explain how the Nested Set model allows for fast retrieval of entire subtrees. What are the performance trade-offs for writes/updates?
4. **Data Warehousing Schemas:** Explain the difference between a Star Schema and a Snowflake Schema in a data warehouse context. What are the advantages and disadvantages of each, particularly regarding query performance vs. data redundancy?
5. **Temporal Data Design:** Design a schema for a table that needs to track the complete history of changes to an employee's salary. You must be able to query what an employee's salary was at any given point in the past.

Pattern 6: Data Transformation & Tricky Scenarios

This category includes problems that require clever thinking and deep knowledge of SQL functions.

Easy

1. **Pivoting with CASE:** You have a sales table with month and amount. Write a query to show total sales for 'Jan', 'Feb', and 'Mar' as columns in a single row.
2. **Handling NULLs:** What is the difference between COUNT(*) , COUNT(column), and COUNT(DISTINCT column)? How does each one handle NULL values?
3. **COALESCE:** A users table has display_name and full_name columns. The display_name is sometimes NULL. Write a query to return the display_name if it exists, otherwise return the full_name.
4. **String Manipulation:** Extract the domain name from a list of user email addresses.
5. **Date Formatting:** Given a DATETIME column, display it in the format 'YYYY-MM-DD'.

Medium

1. **Data Pivoting (Dynamic):** The finance department needs a report showing total sales for each product, with a separate column for each year. You have a sales table with product_name, sale_year, and amount. Write a query to pivot this data.
2. **Cohort Retention Analysis:** To measure product stickiness, you need to perform a cohort analysis. A user's cohort is the month they signed up. For each signup cohort (e.g., '2023-01'), calculate the number of those users who were active in the subsequent months.
3. **Handling NULLs in Aggregations:** You are calculating the average rating for products. Some products have NULL ratings. How does AVG() handle NULLs? How would you write a query to calculate the average rating, treating NULLs as 0? What are the business implications of this choice?
4. **Generating Series:** You need to create a report that shows the total number of new users for every single day in the last month, including days where zero users signed up.
5. **Unpivoting Data:** You have a table with columns product_id, sales_q1, sales_q2, sales_q3, sales_q4. Transform this table so that each row represents a single product for a single quarter (i.e., columns product_id, quarter, sales).

Hard

1. **Monthly Recurring Revenue (MRR):** You have a subscriptions table with user_id, monthly_amount, start_date, and end_date. The end_date can be NULL for active subscriptions. Write a query to accurately calculate the total MRR for each month of the last year, correctly handling subscriptions that start or end mid-month.
2. **Attribution Modeling (Last Touch):** You have a user_touchpoints table with user_id, timestamp, and channel (e.g., 'google', 'facebook'). You also have a conversions table. Write a query to attribute each conversion to the last marketing channel the user interacted with before converting.
3. **Calculating Median:** Write a query to find the median salary for each department without using any built-in MEDIAN or PERCENTILE functions.
4. **Binning / Histograms:** Group users into age buckets (e.g., 20-29, 30-39, 40-49) and count how many users fall into each bucket.
5. **Complex Conditional Logic:** A company has a complex commission rule: 5% on the first $50k of sales in a month, 7.5% on the next $50k, and 10% on anything above $100k. Write a single query to calculate the monthly commission for each salesperson.

Pattern 7: Transactional SQL & Data Integrity

Crucial for Staff-level interviews, these questions test your understanding of how to maintain data consistency in a concurrent environment.

Easy

1. **ACID Properties:** Explain the ACID properties (Atomicity, Consistency, Isolation, Durability) in your own words.
2. **Basic Transaction:** Write the SQL syntax for a simple transaction that inserts a new order and updates the inventory count.
3. **COMMIT vs. ROLLBACK:** What is the difference between COMMIT and ROLLBACK?
4. **Primary Key Constraint:** What is the purpose of a primary key constraint in ensuring data integrity?
5. **NOT NULL Constraint:** Why is the NOT NULL constraint important?

Medium

1. **Isolation Levels:** Explain the difference between READ COMMITTED and REPEATABLE READ isolation levels. Give a simple example of a "non-repeatable read" anomaly.
2. **Foreign Key Constraints:** Explain what ON DELETE CASCADE and ON DELETE SET NULL do. What are the potential dangers of using ON DELETE CASCADE?
3. **CHECK Constraint:** Provide a practical example of when you would use a CHECK constraint on a table column.
4. **Unique Constraint vs. Unique Index:** What is the difference between a UNIQUE constraint and a unique index?
5. **Race Condition:** Describe a simple race condition that could happen in a banking application if transactions are not used correctly (e.g., the "check and then withdraw" problem).

Hard

1. **Preventing Double-Booking:** You are building an inventory system for concert tickets. Multiple users might try to book the last few seats simultaneously. Explain how you would use database transactions and locking (e.g., SELECT ... FOR UPDATE) to prevent the same seat from being sold twice.
2. **Deadlocks:** Describe a specific scenario where a deadlock could occur between two concurrent transactions

updating two tables (e.g., accounts and ledgers). How do databases typically resolve deadlocks? What can you do at the application level to minimize the chance of them occurring (e.g., consistent lock ordering)?

3. **Phantom Reads:** Explain what a "phantom read" is and which isolation level is required to prevent it (SERIALIZABLE). Describe a business scenario where a phantom read could cause incorrect data to be generated (e.g., a reporting query running concurrently with new data insertions).
4. **Optimistic vs. Pessimistic Locking:** Compare and contrast optimistic locking (e.g., using a version column) and pessimistic locking (SELECT FOR UPDATE). When would you choose one approach over the other?
5. **Savepoints:** What is a transaction savepoint? Describe a scenario with a large, multi-step transaction where using savepoints would be beneficial.